

Project Executable Files

Date	02 October 2026
Team ID	SWTID-2026-2606
Project Name	PocketSmartAI
Maximum Marks	3 Marks

PocketSmartAI – Project Executable Files

This section provides the executable and deployable materials for PocketSmartAI, a FastAPI-based smart planning and shopping web application. The application provides user authentication, AI-assisted planning, shopping recommendations, history, and budget-oriented catalog features. The source code is maintained in the Kaviya GitHub repository and the application is deployed as a web service on Render.

Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README/Setup Guide (instructions to run the project)	Yes / Included with project documentation
3	requirements.txt / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes – SQLite database/application database setup
5	Environment configuration file (.env.example or similar)	Required – secrets kept out of GitHub
6	Deployed application URL (if hosted)	Yes – Render
7	APK / Executable binary (if applicable)	NA – Web application
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	To be added if required

10	Demovideoorwalkthrough(ifrequired)	To be added if required
----	------------------------------------	-------------------------

File/Folders Structure Repository Structure: Kavya/

└─
└─
app/
└─
└─
main.py
└─
└─
auth.py
└─
└─
config.py

|
└
—
d
a
t
a
b
a
s
e
.
p
y
|
└
—
m
i
d
d
l
e
w
a
r
e
.
p
y
|
└
—
m
o
d
e
l
s
.

p
y
|
└
—
s
c
h
e
m
a
s
.
p
y
|
└
—
s
e
c
u
r
i
t
y
.
p
y
|
└
—
r
o
u
t
e
s
/

|
|
└
—

a
u
t
h
.
p
y

|
|
└
—

h
i
s
t
o
r
y
.
p
y

|
|
└
—

p
a
g
e
s
.
p
y

|

|
L
—
—

p
l
a
n
n
e
r
s
.
p
y

|
L
—
—

s
e
r
v
i
c
e
s
/

|
└
—
—

c
a
t
a
l
o
g
.
p
y

|
L
—
—
g
e
m
i
n
i
—
s
e
r
v
i
c
e
.
p
y
└
—
—
s
t
a
t
i
c
/
|
└
—
—
c
s
s
/
s
t
y
l
e
.

c
s
s
|
L
—
—
j
s
/
a
p
p
.
j
s
└
—
—
t
e
m
p
l
a
t
e
s
/
|
└
—
—
b
a
s
e
.
h
t
m
l
,
i

[index.html](#), [login.html](#), [register.html](#) | [Feedback](#) - [dashboard](#).

[html](#), [history.html](#) | [Feedback](#) - [home-planner.html](#), [party-pla](#)

n
n
e
r
.
h
t
m
l

|
└
—

j
e
w
e
l
r
y
-
p
l
a
n
n
e
r
.
h
t
m
l
,
s
h
o
p
p
i
n
g
.
h
t
m
l

└
—
—

r
e
q
u
i
r
e
m
e
n
t
s
.
t
x
t

L
—
—

.
g
i
t
i
g
n
o
r
e

M
a
i
n
t
e
c
h
n
o
l
o
g
i

es:
Python,
FastAPI,
Uvicorn,
Jinja2
templates,
SQLite,
py

dependencies, Starlette, and Gemini API.

[Paste or describe your folder structure here. Example:]

- /project-root
- /src - Source code files
- /tests - Test files
- /docs - Documentation
- /config- Configuration files
- README.md- Setup and run instructions
- requirements.txt - Python dependencies (or package.json for Node)

Run Instructions

- 1.Clone or download the Kaviya repository from GitHub.
- 2.Open a terminal in the repository root.
- 3.Create a Python virtual environment:
python -m venv .venv
- 4.Activate it on Windows PowerShell:
.\.venv\Scripts\Activate.ps1
- 5.Install dependencies:
pip install -r requirements.txt
- 6.Configure the required Gemini environment variables locally. Keep API keys private and do not commit them to GitHub.
- 7.Start the application:
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
- 8.Open http://127.0.0.1:8000 in a browser.
- 9.Register/login and test the dashboard, planners, shopping, and history features.

For deployment, Render installs dependencies from requirements.txt and runs the FastAPI application using Uvicorn with the Render-provided PORT.

Pre-requisites:

[List required software, versions, and system requirements]

Step 1: [e.g., Clone the repository: git clone <url>]

Step 2: [e.g., Install dependencies: pip install -r requirements.txt] Step

3: [e.g., Configure environment: copy .env.example to .env] Step 4:

[e.g., Run database migrations: python manage.py migrate] Step 5:

[e.g., Start the application: python manage.py runserver] Step 6:

[e.g., Open browser at http://localhost:8000]

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Render Free web service may sleep after inactivity.	Open the site and allow time for the service to wake up.
2	Gemini features require a valid API key and configured model.	Configure GEMINI_API_KEY and GEMINI_MODEL as secure environment variables.

3	Local setup requires Python dependencies and environment configuration.	Follow the Run Instructions and install requirements.txt.
---	---	---